

LECTURE 24

Paging: Method and Hardware Support

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Paging's basic method — fixed-size frames and pages — and exactly why it eliminates external fragmentation entirely
- The page table and a complete, numeric address-translation walkthrough, bit by bit
- The system-wide frame table
- The Translation Look-aside Buffer (TLB) and a worked Effective Access Time (EAT) calculation
- Protection via the valid-invalid bit and read/write/execute permission bits
- A brief look at hashed and inverted page tables for large address spaces

Paging: Basic Method

- Paging divides physical memory into fixed-size blocks called frames, and a process's logical memory into blocks of the exact same fixed size, called pages
- Any free frame, anywhere in physical memory, can hold any page of any process — pages need not sit next to each other physically

Process logical memory

Page 0, Page 1, Page 2, Page 3 — contiguous in the process's own view

Physical memory

Frame 5, Frame 2, Frame 9, Frame 1 — scattered wherever free frames happen to be

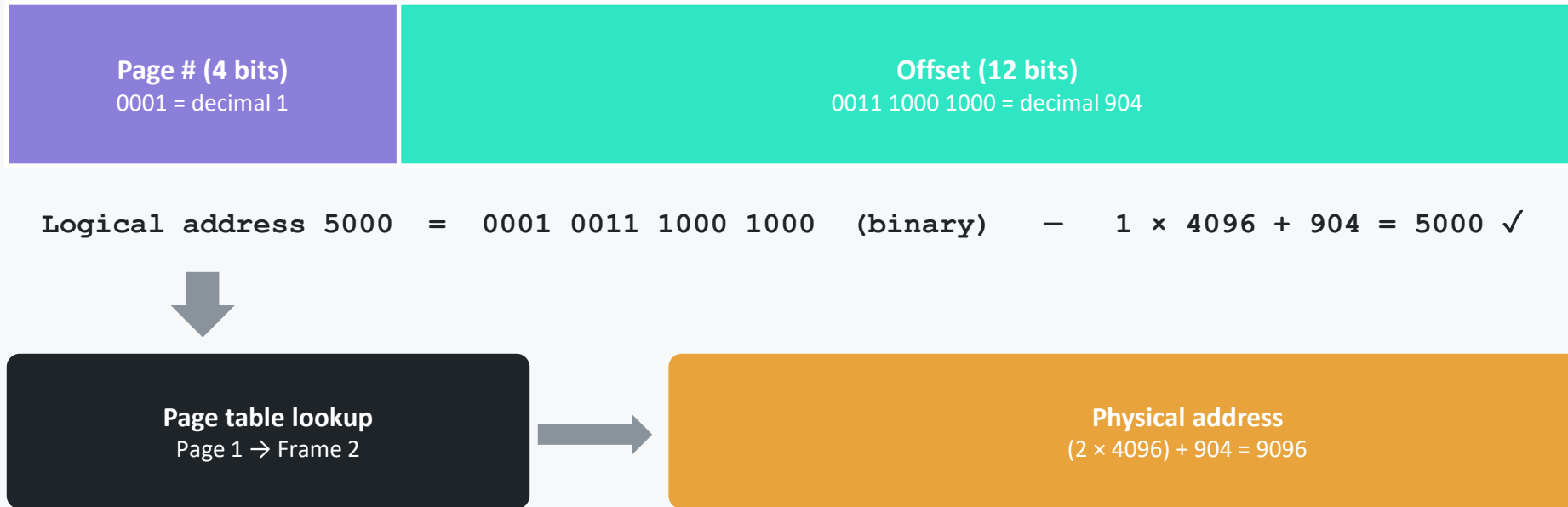
External fragmentation disappears — Every piece is the same fixed size, so a request for n pages only ever needs n free frames, wherever they are. The cost: bounded internal fragmentation in a process's last, typically partially used, page.

The Page Table

- Each process gets its own page table, mapping page number → frame number
- **Page number** — used as an index into this process's page table, yielding the frame number that page currently lives in
- **Offset** — the position within that page/frame — identical whether looking at the logical or the final physical address, since a page and its frame are the same fixed size

Address Translation: A Worked Example

- Page size 4 KB = 4096 bytes = 2^{12} → offset field is 12 bits. Logical address space 64 KB = 2^{16} → $16 - 12 = 4$ bits for the page number (16 pages, 0–15).



The offset never changes — only the page/frame field does. Only the high-order bits (page number) need an actual table lookup; the low-order bits (offset) pass straight through untouched.

Page Table Lookup for This Process

Page number	Frame number
0	5
1	2
2	9
3	1

Page 1 maps to frame 2. The physical address is formed by concatenating the frame number with the same offset computed above:

$$(2 \times 4096) + 904 = 8192 + 904 = 9096$$

The Frame Table

- The OS maintains exactly one system-wide frame table, tracking every physical frame's status and current owner

Frame	Status	Owner (process, page)
0	Free	—
1	Allocated	P7, page 3
2	Allocated	P4, page 1
3	Free	—
4	Allocated	P7, page 0

The frame table is consulted whenever a process needs a new frame (to find a free one), and whenever a frame needs reclaiming (to know who to notify).

Hardware Support: The TLB

- A naive implementation doubles every memory access: one to read the page table entry, one to fetch the data
- The Translation Look-aside Buffer (TLB) is small, fast associative-memory hardware caching recent page→frame translations, checked before the full page table

TLB hit

The needed translation is already cached; the frame number comes back almost immediately, and the full page table is never touched

TLB miss

The translation isn't cached; the CPU walks the real page table (one extra memory access), then caches the result before accessing the data

Effective Access Time (EAT)

$$\text{EAT} = \text{hit_ratio} \times (\text{TLB time} + \text{memory time}) + \text{miss_ratio} \times (\text{TLB time} + 2 \times \text{memory time})$$

Worked example: TLB access takes 10 ns, a main-memory access takes 100 ns, and the TLB hit ratio is 80% (0.8), so the miss ratio is 20% (0.2):

$$\begin{aligned}\text{EAT} &= 0.8 \times (10 + 100) + 0.2 \times (10 + 2 \times 100) \\ &= 0.8 \times 110 + 0.2 \times (10 + 200) \\ &= 88 + 0.2 \times 210 \\ &= 88 + 42 \\ &= 130 \text{ ns}\end{aligned}$$

Hit ratio is the whole game — Every term is fixed by the hardware except the hit ratio, which depends on program locality. A higher hit ratio drives EAT closer to a single memory access.

Protection in Paging

- **Valid-invalid bit** — marks whether a page is actually part of the process's legitimate address space right now. Accessing an invalid page traps to the OS immediately.
- **Read/write/execute permission bits** — independently restrict what kind of access is allowed per entry.

Page	Frame	Valid	Read	Write	Execute
0 (code)	5	Yes	Yes	No	Yes
1 (heap)	2	Yes	Yes	Yes	No
2 (unused)	—	No	—	—	—

Advanced: Hashed and Inverted Page Tables

- **Hashed page tables** — the page number is hashed into a much smaller table; each entry holds the page number (to resolve collisions) and frame number. Handles large, sparse address spaces efficiently.
- **Inverted page tables** — one table, system-wide, with one entry per physical frame, recording which process and page occupies it. Saves enormous memory, at the cost of a slower lookup.

Same trade-off, different scale — Both trade some lookup speed for a dramatic reduction in page-table memory — the right choice when a process's address space is far larger and sparser than its actual usage.

Key Takeaways

- Paging divides memory into fixed-size frames and pages — any free frame fits any page, eliminating external fragmentation (at the cost of bounded internal fragmentation)
- A page table maps page number → frame number; translation splits the address into page-number and offset fields, looks up the frame, and concatenates frame + offset
- The frame table is one system-wide structure tracking every physical frame's allocation status and owner
- The TLB caches recent translations; EAT blends hit and miss cost — an 80% hit ratio with 10 ns TLB and 100 ns memory gave $EAT = 130$ ns
- Protection comes from the valid-invalid bit and read/write/execute permission bits per entry
- Hashed page tables hash the page number for sparse spaces; inverted page tables keep one entry per physical frame, trading lookup speed for memory